Data Models

Enforcement

Tarapong Sreenuch

8 February 2026

克明峻德，格物致知

Building On: Topics 2–4

Topics 2–4 produced a correct, normalised, queryable schema:

- **Topic 2:** ERD → structurally correct schema (PKs, FKs, cardinality constraints)
- **Topic 3:** BCNF normalisation eliminated redundancy; justified downshifts documented
- **Topic 4:** FWGHOS pipeline queries the 14-table Orders & Payments schema correctly

The schema is sound. The queries are correct. But the database will accept **any data you throw at it**, let **anyone see everything**, and leave **half-finished operations visible** after a crash.

The Orders & Payments Schema: Deployed to Production

The team deploys the schema to production. Three things go wrong in the first month.

```
customers ──< orders ──< order_lines >── products
                │
                ├──── payments
                └──── shipments
```

Primary keys and foreign keys are in place. The structure is correct. Every query from Topic 4 returns the right data.

So what goes wrong?

Failure 1: The SGD 12,000 Duplicate Invoice

A batch import script loads 10,000 orders. Two orders for the same customer arrive with identical invoice numbers:

```
orders
order_id | customer_id | invoice_number    | status
---------+-------------+-------------------+----------
101      | 1           | INV-2024-00101    | completed
110      | 1           | INV-2024-00101    | pending     ← duplicate
```

The schema accepts both. The accounting system sends two payment requests for SGD 34.97 each. Alice's card is charged twice — SGD 69.94 instead of SGD 34.97. Alice disputes the second charge.

Direct cost: refund processing, staff time, and a customer who no longer trusts the system.

Failure 1: Root Cause

The root cause is not the batch script. The root cause is that the schema has **no rule** saying "each customer's invoice numbers must be unique."

The application that generates invoice numbers has a uniqueness check — but the batch import script bypasses the application entirely. It writes directly to the database.

Any enforcement that lives only in one application is enforcement that every other access path ignores.

Failure 2: The Analyst Who Saw Everything

A reporting analyst needs product revenue figures. The DBA grants `SELECT` on **all tables**.

```
reporting_user ──SELECT──▶ customers  (emails, postcodes)
               ──SELECT──▶ orders     (statuses, dates)
               ──SELECT──▶ payments   (amounts, methods)
               ──SELECT──▶ order_lines
               ──SELECT──▶ products
```

The analyst exports customer payment details to a shared spreadsheet. The company is now in breach of its data handling policy.

Failure 2: Root Cause

The root cause is not the analyst's curiosity. The root cause is that the database **exposes everything to everyone**.

There is no mechanism to say "this user can see product revenue but not customer emails."

The DBA's shortcut — grant everything, sort it out later — means the only protection is **hoping users choose not to look**.

Failure 3: Carol's Double Charge

Carol completes checkout. The application executes two statements:

```
-- Statement 1: record the payment
INSERT INTO payments VALUES (207, 106, 39.95, '2024-11-20', 'card');
-- auto-committed ✓

-- ╳ APPLICATION CRASH ╳

-- Statement 2: mark order as paid
UPDATE orders SET status = 'paid' WHERE order_id = 106;
-- never runs
```

Payment 207 exists. Order 106 still shows `'pending'`. Carol sees "pending," clicks "pay" again. A second payment is inserted.

Carol is charged SGD 79.90 for a SGD 39.95 order.

Failure 3: Root Cause

The root cause is not the crash. Crashes happen.

The root cause is that two statements that must succeed together were **executed independently**. Each statement was individually valid — the INSERT satisfied all FK constraints, the UPDATE would have been valid too.

The problem is the **gap between them**: a moment where the database is in a state that no business rule allows.

Three Failures: One Root

	Failure 1	Failure 2	Failure 3
Symptom	Duplicate invoices (SGD 12K)	Analyst sees customer emails	Double charge (SGD 79.90)
Root cause	No UNIQUE constraint	No access restriction	No transaction boundary
What's correct	Structure, FKs, queries	Structure, FKs, queries	Structure, FKs, queries
What's missing	Integrity enforcement	Access control	Atomicity

Predict: The tables are in BCNF, the FKs are correctly placed, the queries return the right data. What must the schema enforce — beyond structure — to prevent these failures?

The Common Thread

All three failures share one root: the schema has correct **structure** but incomplete **enforcement**.

Structure describes what the building should look like. Enforcement is the building code that prevents someone from removing a load-bearing wall.

> Therefore, the engineer must build enforcement into the **schema itself** — not into application code, not into policy documents, not into hope.

Four Required Capabilities

Each capability maps directly to a failure:

Capability	Failure prevented	Test
1. Integrity enforcement	Failure 1 — duplicate invoices	INSERT duplicate; observe rejection
2. Safe data exposure	Failure 2 — unauthorised access	Query view (succeeds); query base table (denied)
3. Atomic multi-step changes	Failure 3 — double charge	Crash after INSERT; verify ROLLBACK
4. Read performance at scale	All queries degrade as data grows	EXPLAIN shows index usage

Requirements 1–3 are **correctness**. Requirement 4 is **performance**.

The Dependency Chain

The four capabilities are not independent — each layer depends on the previous:

```
Layer 4: Indexes ──────────── speed over correct, secure, atomic data
      │
Layer 3: Transactions ─────── atomic changes to access-controlled data
      │
Layer 2: Views + GRANT ────── controlled exposure of constrained data
      │
Layer 1: Constraints ──────── reject invalid data at write time
      │
Foundation: Schema Structure (Topics 2–4)
```

Predict: Why must constraints come before views, and views before transactions? What breaks if you add indexes first and constraints last?

The Constraint Hierarchy: Five Levels of "No"

Each constraint type asks a more specific question about the data:

PRIMARY KEY	UNIQUE + NOT NULL; row identity
FOREIGN KEY	Value must exist in referenced table
UNIQUE	No duplicate values on column set
CHECK	Value must satisfy a domain predicate
NOT NULL	Column must have a value

▲ Each level answers a more specific question about the data

Topics 2–4 already placed PRIMARY KEY and FOREIGN KEY. This topic adds **CHECK**, **UNIQUE**, and **NOT NULL**.

What the Unconstrained Schema Permits

The valid baseline — one sample row per table:

```
customers:    (1, 'Alice Tan', 'alice@example.com', '018956')
orders:       (101, 1, 'INV-2024-00101', '2024-03-15', 'completed', 34.97)
order_lines:  (101, 1, 2, 9.99)          — Widget A ×2
products:     (1, 'Widget A', 9.99, 500)
payments:     (201, 101, 34.97, '2024-03-15', 'card')
```

Now consider three bad inserts that should fail but **don't**:

Bad insert	What is wrong	Constraint needed
`INSERT INTO order_lines VALUES (101, 6, -3, 199.99)`	Negative quantity	`CHECK (quantity > 0)`
`INSERT INTO payments VALUES (208, 101, 0.00, ... )`	Zero-amount payment	`CHECK (amount > 0)`
`INSERT INTO orders VALUES ... 'INV-2024-00101' ...`	Duplicate invoice	`UNIQUE (customer_id, invoice_number)`

Bad Data Accepted: No Error, No Warning

All three inserts succeed. A Screen 24" with quantity **-3** is now stored on Alice's order:

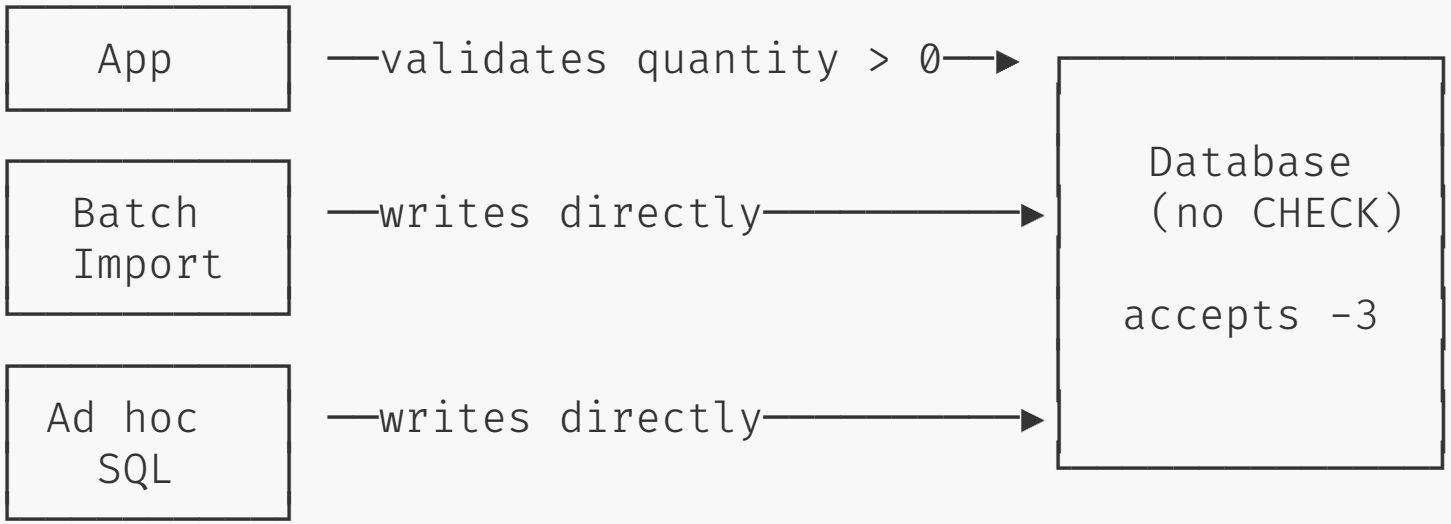
```
order_lines (after bad insert)
order_id | product_id | quantity | unit_price
---------+------------+----------+----------
101      | 1          | 2        | 9.99
101      | 6          | -3       | 199.99     ← impossible value
```

The schema's structure is correct — composite key is valid, foreign keys match. But the **data** is impossible.

What does the absence of a CHECK constraint mean for batch imports, ad hoc SQL sessions, and future API endpoints?

Every Access Path Bypasses Application Checks

Any client that can connect to the database can write impossible data:



The only enforcement point that covers **every** access path is the schema itself.

Adding the Constraints

Each constraint traces to a named business rule:

```
-- Integrity: quantities must be positive
ALTER TABLE order_lines ADD CONSTRAINT chk_quantity_positive
    CHECK (quantity > 0);

-- Integrity: payments must be positive
ALTER TABLE payments ADD CONSTRAINT chk_amount_positive
    CHECK (amount > 0);

-- Integrity: order status is a defined set
ALTER TABLE orders ADD CONSTRAINT chk_status_valid
    CHECK (status IN ('pending','paid','shipped','completed','cancelled'));

-- Uniqueness: one invoice number per customer
ALTER TABLE orders ADD CONSTRAINT uq_customer_invoice
    UNIQUE (customer_id, invoice_number);

-- Uniqueness: each customer email must be unique
ALTER TABLE customers ADD CONSTRAINT uq_customer_email
    UNIQUE (email);

-- Mandatory fields
ALTER TABLE orders MODIFY order_date DATE NOT NULL;
ALTER TABLE orders MODIFY status VARCHAR(20) NOT NULL;
```

Before vs After: The Same Three Inserts

Before constraints — all three succeed:

```
INSERT INTO order_lines VALUES (101, 6, -3, 199.99);
-- Query OK, 1 row affected

INSERT INTO payments VALUES (208, 101, 0.00, '2024-11-20', 'card');
-- Query OK, 1 row affected
```

After constraints — all three rejected:

```
INSERT INTO order_lines VALUES (101, 6, -3, 199.99);
-- ERROR 3819 (HY000): Check constraint 'chk_quantity_positive' is violated.

INSERT INTO payments VALUES (208, 101, 0.00, '2024-11-20', 'card');
-- ERROR 3819 (HY000): Check constraint 'chk_amount_positive' is violated.

INSERT INTO orders (order_id, customer_id, invoice_number, order_date, status)
VALUES (110, 1, 'INV-2024-00101', '2024-11-01', 'pending');
-- ERROR 1062 (23000): Duplicate entry '1-INV-2024-00101' for key 'uq_customer_invoice'
```

One line of DDL changed the schema from "accepts anything" to "rejects impossible data."

FK Cascade Behaviours: When the Parent Goes Away

Action	Effect	When to use
RESTRICT (default)	Block delete if children exist	Independent entities — `customers` → `orders`
CASCADE	Delete all children	Weak entities — `orders` → `order_lines`
SET NULL	Set children's FK to NULL	Optional relationships — `employees` → department

The FK cascade chain for Orders & Payments:

```
customers ──RESTRICT──▶ orders ──CASCADE──▶ order_lines
                          │
                          └──RESTRICT──▶ payments
```

RESTRICT on `customers` → `orders`: deleting a customer must not silently destroy their entire order history.

> **Predict:** Topic 2's ERD showed `order_lines` as a weak entity — composite key includes `order_id`. The current FK uses the default RESTRICT. What happens if you try to DELETE an order that has line items?

CASCADE vs RESTRICT: Revealed

> The delete is **blocked**: "Cannot delete or update a parent row." RESTRICT prevents the delete because child rows reference this order.

```
DELETE FROM orders WHERE order_id = 101;
-- ERROR 1451 (23000): Cannot delete or update a parent row:
-- a foreign key constraint fails (`payments`, CONSTRAINT `fk_payments_orders`)
```

To delete the order and its line items together, use the **DROP/ADD FK pattern** to replace the existing FK with a cascading version:

```
ALTER TABLE order_lines
  DROP FOREIGN KEY fk_order_lines_orders,
  ADD CONSTRAINT fk_order_lines_orders
    FOREIGN KEY (order_id) REFERENCES orders(order_id)
    ON DELETE CASCADE;
```

Tip: If the FK has a different name (e.g., MySQL's auto-generated `order_lines_ibfk_1`), run `SHOW CREATE TABLE order_lines` to find the actual constraint name before DROP.

Appropriate because line items have no independent existence. But CASCADE on `customers` → `orders` would silently destroy entire order histories — almost certainly wrong.

Why Constraints Belong in the Schema

Approach	Enforcement Point	Coverage	Failure mode
Application code	One application	That application only	Any other access path bypasses
Database constraint	The schema	Every write, regardless of source	None — engine rejects universally
Manual review	Human	Depends on attention	Errors slip through under pressure

The database is the single source of truth — so it must be the single enforcement point. Application checks complement schema constraints; they do not replace them.

But rejecting invalid data is only half the problem. The database still exposes **everything** to **everyone** — Failure 2.

Views: Named Queries for Safe Exposure

A view is a saved query with a name. Three reasons to use them:

- **Simplification** — wrap a multi-table join; users query `SELECT * FROM revenue_by_product`
- **Restriction** — expose only the columns and rows needed for a specific role
- **Stability** — if underlying tables change, update the view; users are insulated

```
CREATE VIEW customer_orders AS
SELECT c.name AS customer_name,
       COUNT(o.order_id) AS order_count,
       COALESCE(SUM(o.order_total), 0) AS total_spend
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name;
```

The LEFT JOIN ensures customers with zero orders still appear (Eve Santos: 0 orders, SGD 0.00).

What the View Shows vs What It Hides

What `customer_orders` shows	What the base tables contain (hidden)
customer_name	email, postcode, sales_rep_id
order_count	order_date, status, invoice_number, shipping details
total_spend	payment amounts, payment methods

```
customer_orders (view output — 3 of 5 rows shown)
customer_name | order_count | total_spend
──────────────┼─────────────┼────────────
Alice Tan     | 2           | 84.96
Dave Wong     | 1           | 29.96
Eve Santos    | 0           | 0.00
```

Cost: A view stores no data — the underlying SELECT re-runs every time. Heavy aggregations on large tables carry a per-query performance cost. MySQL does not support materialised views natively.

The `revenue_by_product` View: Simplification

The second view encapsulates a multi-table join for product revenue:

```
CREATE VIEW revenue_by_product AS
SELECT p.name AS product_name,
       SUM(ol.quantity) AS total_quantity_sold,
       SUM(ol.quantity * ol.unit_price) AS total_revenue
FROM products p
JOIN order_lines ol ON p.product_id = ol.product_id
GROUP BY p.product_id, p.name;
```

The **INNER JOIN** means products with zero sales are excluded — Screen 24" has no order lines, so it does not appear. This is the same silent-data-loss pattern from **Topic 4's Failure 2**. Intentional here: the revenue view reports on products that *have* sales. A LEFT JOIN version would show all 6 products with Screen 24" at SGD 0 revenue.

`revenue_by_product` Output: Five Rows

```
revenue_by_product (seed data — Screen 24" excluded by INNER JOIN)
product_name | total_quantity_sold | total_revenue
──────────────┼────────────────────┼────────────────
Widget A     | 6                  | 59.94
Widget B     | 3                  | 44.97
Gadget X     | 2                  | 99.98
Gadget Y     | 2                  | 159.98
Cable USB-C  | 6                  | 29.94
```

Five of six products appear. Screen 24" is absent — no order lines reference it, so the INNER JOIN drops it. Compare with a LEFT JOIN: all 6 products would appear, with Screen 24" showing 0 quantity and SGD 0.00 revenue.

GRANT, REVOKE, and the Trust Boundary

The pattern for safe data exposure — three steps:

1. **Create a view** that shapes data for a specific role
2. **Create a database user** for that role
3. **GRANT SELECT on the view** — do not grant anything on base tables

```
CREATE USER 'reporting_user'@'localhost' IDENTIFIED BY '...';
GRANT SELECT ON orders_payments.customer_orders TO 'reporting_user'@'localhost';
GRANT SELECT ON orders_payments.revenue_by_product TO 'reporting_user'@'localhost';
-- No GRANT on base tables
```

The user queries the view but **cannot see or modify** base tables. The view is the trust boundary; GRANT enforces it.

The Trust Boundary in Action

```
reporting_user ──▶ ┌──────────────────┐ ──▶ SELECT √
                   │ customer_orders  │
                   │ revenue_by_prod  │ ──▶ SELECT √
                   └──────────────────┘
                          │
═══════════════════════════╪═══════════ trust boundary
                          │
                   ┌──────────────────┐ ──▶ DENIED ✗
                   │ customers        │
                   │ orders           │ ──▶ DENIED ✗
                   │ payments         │ ──▶ DENIED ✗
                   │ products         │ ──▶ DENIED ✗
                   └──────────────────┘
```

Predict: A `reporting_user` has SELECT on the `revenue_by_product` view but no permissions on base tables. Can they see customer emails? Can they run `UPDATE products SET price = 0`?

Trust Boundary: Revealed

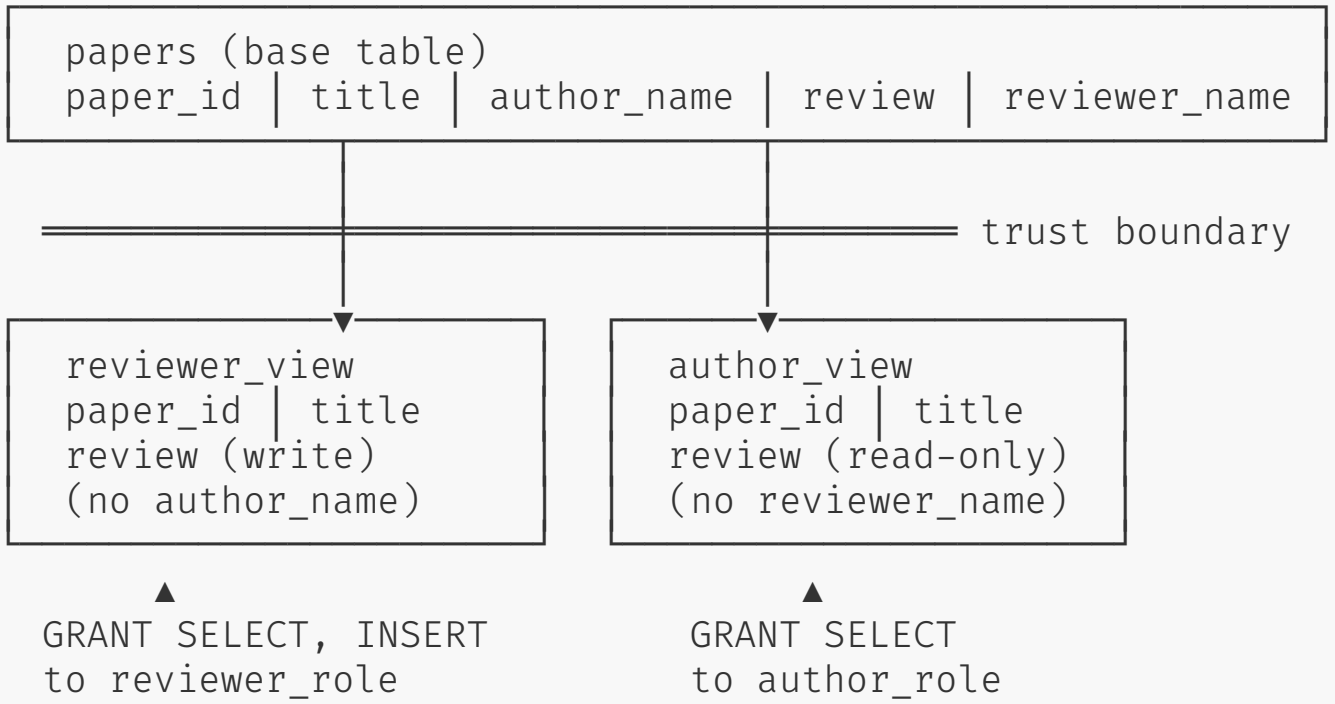
> No to both. The view does not include customer emails — it joins `products` and `order_lines`, not `customers`. The user has no UPDATE permission on any object. Even `SELECT * FROM customers` returns "permission denied."

The permission matrix:

Operation	reporting_user	app_user	admin
SELECT on views	Yes	No (uses base tables)	Yes
SELECT on base tables	**Denied**	Yes	Yes
INSERT on base tables	**Denied**	Yes	Yes
UPDATE on base tables	**Denied**	Yes	Yes
DELETE on base tables	**Denied**	**Denied**	Yes
CREATE/ALTER/DROP	**Denied**	**Denied**	Yes

The Pattern Generalised: Double-Blind Review

The same view + GRANT pattern applies beyond reporting. A double-blind review system:



Neither role sees the full base table. The view defines visibility; GRANT enforces it.

Data is now constrained and access-controlled. But Failure 3 — Carol's double charge — is still possible.

Transactions: All or Nothing

A transaction wraps multiple statements into a single unit of work: `**BEGIN**` opens it, `**COMMIT**` makes all changes permanent at once, `**ROLLBACK**` discards all changes since BEGIN.

Property	Meaning
Atomicity	All operations succeed, or none do — prevents Failure 3
Consistency	Database moves from one valid state to another — links constraints to transactions
Isolation	Concurrent transactions don't interfere
Durability	Committed changes survive crashes

This topic focuses on **atomicity** and **consistency** — the properties that directly address Failure 3.

Auto-Commit vs Transaction-Wrapped

```
Auto-commit (dangerous)

INSERT payments 207
─── committed ✓ ───
      │
   ╳ CRASH ╳
      │
UPDATE orders status
─── never runs ───


Result:
• payments 207 exists
• order 106 still 'pending'
• Carol charged SGD 39.95
• INCONSISTENT
```

```
Transaction-wrapped (safe)

 BEGIN
│  INSERT payments 207
│  ─── pending ───
│       │
│    ╳ CRASH ╳
│       │
│  ROLLBACK (automatic)
│  ─── all discarded ───

Result:
• No payments 207
• order 106 still 'pending'
• Carol not charged
• CONSISTENT
```

State Trace: Auto-Commit vs Transaction

The same crash, two different outcomes:

Step	Auto-commit state	Transaction state
Before	`payments`: none for order 106; `orders` 106: status = 'pending'	Same
After INSERT `payments`	payment 207 **committed**	payment 207 **uncommitted**
Crash occurs	—	—
After recovery	payment 207 exists; order 106 still 'pending' — **inconsistent**	payment 207 rolled back; order 106 still 'pending' — **consistent**
Carol sees	"Charged SGD 39.95 but order shows unpaid" — clicks pay again	"Order still pending" — pays once correctly

Auto-commit makes each statement permanent immediately. The transaction defers permanence until COMMIT — so a crash before COMMIT discards everything.

The Complete Transaction Pattern

```
BEGIN;

-- Step 1: record the payment
INSERT INTO payments (payment_id, order_id, amount, payment_date, method)
VALUES (207, 106, 39.95, '2024-11-15', 'card');

-- Step 2: mark the order as paid
UPDATE orders SET status = 'paid' WHERE order_id = 106;

-- Safety check: did the UPDATE find the right order?
-- If ROW_COUNT() = 0, the order_id was wrong → ROLLBACK

COMMIT;
```

If any step fails — constraint violation, wrong order_id, crash — `ROLLBACK` discards all pending changes. The trigger maintaining `order_total` also fires inside this transaction: if the transaction rolls back, the trigger's recalculation is rolled back too.

> **Predict:** Two statements without a transaction: INSERT a payment for order 106 (succeeds, auto-commits), then UPDATE orders SET status = 'paid' WHERE order_id = 9999 (fails — order 9999 doesn't exist). What state is the database in?

The Gap Between Valid Statements

> The payment row for order 106 exists (committed), but order 106's status remains 'pending'. The database is structurally valid — no constraint is violated — but semantically wrong.

Constraints reject invalid **individual** writes. Transactions ensure the database moves from one valid state to another **without exposing the in-between**.

The division of labour: **constraints** define what "valid" means, **access control** restricts who can write, and **transactions** ensure no intermediate state is visible.

Isolation Levels: Reference

Level	Dirty Reads	Non-Repeatable Reads	Phantom Reads
READ UNCOMMITTED	Possible	Possible	Possible
READ COMMITTED	Prevented	Possible	Possible
REPEATABLE READ (MySQL default)	Prevented	Prevented	Possible*
SERIALIZABLE	Prevented	Prevented	Prevented

* MySQL's InnoDB uses gap locks that prevent most phantoms in practice; verify behaviour with your engine.

Isolation and durability matter in production — but this topic's focus is **atomicity** and **consistency**: the properties that directly address Failure 3.

Correctness is now covered — constraints, access control, transactions. The final capability: **read performance at scale**.

Indexes: Speed Without Compromise

Without an index, the database reads **every row** to find matches. At 1,000 rows, you don't notice. At 1,000,000, the same query takes seconds.

An index is a separate data structure — usually a **B-tree** — that maps column values to row locations.

Cost: Every INSERT or UPDATE also updates every index on that table. For write-heavy tables, unnecessary indexes slow every write.

The key concept is **selectivity** — how many rows match a given predicate:

Selectivity	Example	Index Benefit
High (few match)	`WHERE customer_id = 4` (~14% of 100K rows)	Large — skips ~86,000 rows
Medium	`WHERE order_date BETWEEN ...`	Moderate
Low (many match)	`WHERE status = 'completed'` (57% of rows)	Minimal — optimiser may ignore

Full Table Scan vs B-Tree Lookup

```
Full table scan (no index)                B-tree index lookup
100,000 rows                              ~17 comparisons

┌──────────────────────────┐
│ row 1: customer_id = 1   │ ← read
│ row 2: customer_id = 3   │ ← read
│ row 3: customer_id = 1   │ ← read
│ row 4: customer_id = 4   │ ← read ★        [50000]  ← root
│ row 5: customer_id = 2   │ ← read              │
│ ...                      │ ← read              │ 4 < 50000 → go left
│ row 99,999               │ ← read              ▼
│ row 100,000              │ ← read          [25000]
└──────────────────────────┘                      │
                                                  │ 4 < 25000 → go left
Reads: 100,000 rows                               ▼
Finds: ~14,000 matches                     ... 15 more steps ...
                                                  │
                                                  ▼
                                            row 4 ★   → direct jump
                                            row 48 ★  → next leaf

                                          Reads: ~14,000 matches
```

Double the table size → add one more comparison. That's logarithmic growth.

EXPLAIN: The Diagnostic Tool

Prefix any SELECT with `EXPLAIN` to see the query plan:

Before index:

```
type: ALL    rows: 100,000    Extra: Using where
```

`ALL` = full table scan. No index used.

After `CREATE INDEX idx_orders_customer ON orders(customer_id)`:

```
type: ref    rows: 14,286    Extra: NULL
```

`ref` = index lookup. Rows examined dropped from 100,000 to 14,286 — an **86% reduction**.

> **Predict:** Two queries on 100K orders — Query A: `WHERE customer_id = 4` (~14% of rows). Query B: `WHERE status = 'completed'` (~57% of rows). Which benefits more from an index?

Selectivity Determines Index Value

> Query A benefits dramatically — the index skips ~86,000 rows. Query B benefits minimally — the engine still reads 57% of the table. The optimiser may choose to ignore the index entirely for Query B.

Index decisions for Orders & Payments:

Column	Used In	Selectivity	Index?
`orders.customer_id`	WHERE, JOIN	High	**Yes**
`orders.order_date`	WHERE, ORDER BY	Medium	**Yes**
`order_lines.product_id`	JOIN (views)	High	**Yes**
`orders.status`	WHERE	Low (57%)	**No**
`customers.customer_id`	—	—	Already indexed (PK)

Triggers: Closing the Loop on Derived Attributes

Topic 3's justified downshift: `order_total` on the `orders` table — a derived attribute. The safety mechanism was stated but never built. Now we build it.

```
DELIMITER //
CREATE TRIGGER trg_orderline_after_insert
AFTER INSERT ON order_lines
FOR EACH ROW
BEGIN
    UPDATE orders
    SET order_total = (
        SELECT SUM(quantity * unit_price)
        FROM order_lines
        WHERE order_id = NEW.order_id
    )
    WHERE order_id = NEW.order_id;
END //
DELIMITER ;
```

The trigger fires for **every** write — batch imports, ad hoc SQL, REST API. Single maintenance point.

Cost: Triggers are invisible to the writer of the INSERT. Debugging requires checking the database, not just application code. The SUM runs on every insert, adding write latency.

Trigger Demonstration

A corresponding AFTER DELETE trigger (same recalculation pattern) also maintains `order_total` when line items are removed.

```
Order 106 — before and after line item changes:

Step                                order_total
──────────────────────────────────────────────
Before                              SGD 39.95
INSERT (Widget B ×1, SGD 14.99)     SGD 54.94   ← trigger recalculated
DELETE (Widget B line)              SGD 39.95   ← trigger recalculated
```

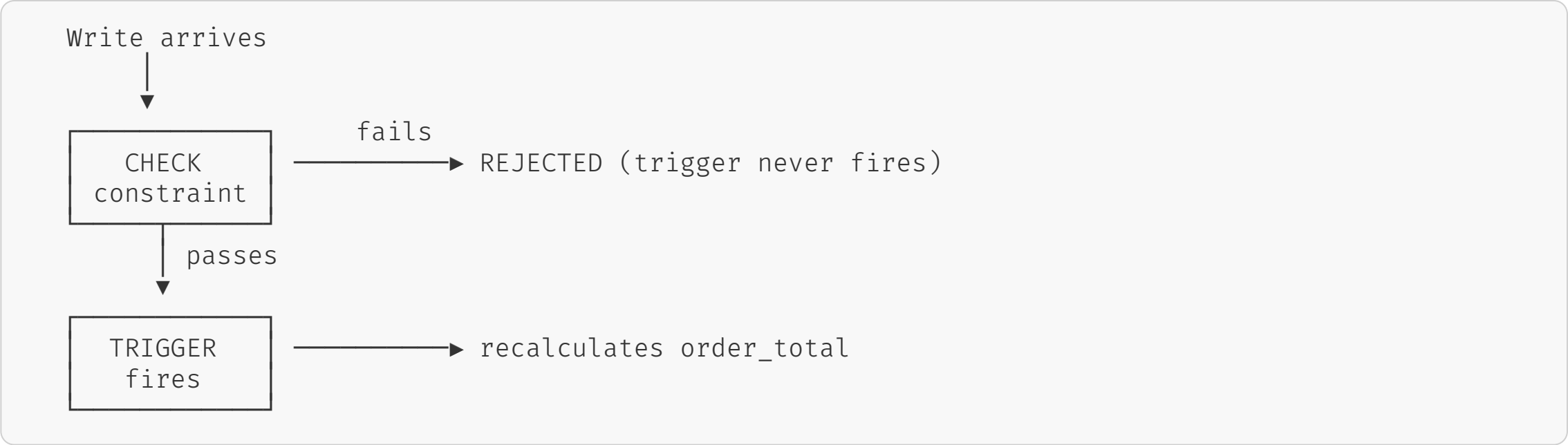
No application code ran. No stored procedure was called. The trigger fired automatically on both the INSERT and the DELETE.

What happens if someone inserts a line item with `quantity = -2`? The CHECK constraint rejects it — but does the trigger fire?

Constraint + Trigger Interaction

> No — the CHECK constraint rejects the INSERT **before** the trigger fires. The `order_total` remains unchanged because no row was written.

Constraints and triggers work in sequence:



The trigger only ever recalculates from **valid** data. The trigger also runs within the same transaction — if the transaction rolls back, the trigger's UPDATE is rolled back too.

The Enforcement Stack: All Four Layers

```
┌──────────────────────────────────────────────────────────┐
│ Layer 4: INDEXES          Accelerate reads over correct,  │
│                           secure, atomic data             │
├──────────────────────────────────────────────────────────┤
│ Layer 3: TRANSACTIONS     Multi-step operations           │
│                           all-or-nothing                  │
├──────────────────────────────────────────────────────────┤
│ Layer 2: VIEWS + GRANT    Each role sees only             │
│                           what it needs                   │
├──────────────────────────────────────────────────────────┤
│ Layer 1: CONSTRAINTS      Reject invalid data             │
│          + TRIGGERS       at write time                   │
├──────────────────────────────────────────────────────────┤
│ Foundation: Schema Structure (Topics 2–4)                 │
└──────────────────────────────────────────────────────────┘
```

Each layer depends on the one below. Removing any layer weakens the layers above it.

Callback: The Dependency Chain Answered

Earlier we asked: why must constraints come before views, and views before transactions?

- Without **constraints**, views expose garbage data, transactions protect meaningless invariants, and indexes speed up wrong answers
- Without **views + GRANT**, constrained data is exposed to everyone — Failure 2
- Without **transactions**, constrained and access-controlled data can be left in inconsistent states — Failure 3
- Without **indexes**, correct, secure, atomic data is correct but slow to retrieve as it grows

The ordering is a dependency chain, not a preference.

The Eight-Step Hardening Procedure

```
┌─ Layer 1: Constraints ──────────────────────────────────────
│  Step 1: Add CHECK, UNIQUE, NOT NULL
│  Step 2: Test each constraint with bad data
│  Step 3: Choose FK cascade behaviours
├─ Layer 2: Access Control ───────────────────────────────────
│  Step 4: Create reporting views
│  Step 5: Create users, GRANT least privilege
├─ Layer 3: Atomicity ────────────────────────────────────────
│  Step 6: Wrap multi-step operations in transactions
├─ Layer 4: Performance ──────────────────────────────────────
│  Step 7: Add indexes for high-selectivity predicates
├─ Integration ───────────────────────────────────────────────
│  Step 8: Verify the full stack end to end
└─────────────────────────────────────────────────────────────
```

Each constraint must trace to a **named business rule**. If you can't name the rule, don't add the constraint.

Check 1: Constraint Rejection (Library)

A city library's `book_loans` table has constraints: `CHECK (loan_days > 0)`, `UNIQUE (member_id, isbn, loan_date)`, and `FK member_id REFERENCES members(member_id)`.

Three bad inserts:

```
INSERT INTO book_loans VALUES (1, 'ISBN-001', '2024-09-01', -5);
INSERT INTO book_loans VALUES (9999, 'ISBN-002', '2024-09-01', 7);
INSERT INTO book_loans VALUES (1, 'ISBN-001', '2024-09-01', 14);
```

What happens to each? What does it mean if any succeeds?

> All three are rejected — CHECK violated, FK violated, UNIQUE violated. If any succeeds, the constraint is missing or wrong: the ALTER TABLE didn't run, the CHECK syntax is incorrect (MySQL before 8.0.16 silently ignores CHECK), or the UNIQUE was added on the wrong column set. The test is the evidence — same principle as Orders & Payments.

Check 2: CASCADE vs RESTRICT (Airline)

An airline's schema: `flights —< bookings —< passengers`. The FK from `bookings` to `flights` uses RESTRICT (default).

A cancelled flight (FL-207) has 180 bookings. Delete the flight row.

Does the delete succeed?

> **No** — RESTRICT blocks it because 180 booking rows reference this flight. Change the FK to CASCADE and repeat: the flight row **and all 180 bookings and their passenger records** are deleted. The cascade propagates through the FK chain. A cancelled flight needs rebooking, not silent deletion. Correct pattern: RESTRICT on `flights` → `bookings`, CASCADE on `bookings` → `passengers` (weak entity).

Check 3: View + GRANT Isolation (Banking)

A bank creates an `account_summary` view for tellers — shows account holder name, account type, and balance. The base `accounts` table also contains SSN, internal risk score, and credit limit.

`teller_user` has SELECT on `account_summary` only. Connect as `teller_user`:

```
SELECT * FROM account_summary;        -- ?
SELECT * FROM accounts;               -- ?
UPDATE accounts SET balance = 0;      -- ?
```

> First query: **succeeds** (account holder, type, balance). Second query: **denied** (no permission on base table — SSN and risk scores hidden). Third query: **denied** (no UPDATE permission on any object). The trust boundary is the view; GRANT enforces it — same pattern as Orders & Payments.

Check 4: Transaction Rollback (Ride-Sharing)

A ride-sharing app completes a trip. BEGIN → INSERT `ride_payments` (trip 4821, SGD 23.50) → UPDATE `driver_balances` with `driver_id = 9999` (non-existent — UPDATE affects 0 rows, no error).

Is the ride payment visible?

> The zero-row UPDATE doesn't automatically roll back the transaction — it's still open with the INSERT applied. The developer must check `ROW_COUNT()`, detect the wrong driver_id, and issue `ROLLBACK`. After ROLLBACK, the ride payment is gone — the driver is not credited. Without the transaction (auto-commit), the INSERT would have been permanent — a payment exists with no corresponding driver credit.

Check 5: Index Selectivity (Hospital)

A hospital's `appointments` table has 500,000 rows. Two queries:

```
-- Query A: one doctor's appointments (~2% of rows)
SELECT * FROM appointments WHERE doctor_id = 42;

-- Query B: completed appointments (~70% of rows)
SELECT * FROM appointments WHERE status = 'completed';
```

Which benefits from an index?

> Query A — dramatically. The index skips ~490,000 rows. EXPLAIN shows `type: ref`. Query B — minimally. The engine reads 70% of the table. The optimiser may ignore the index entirely. **Selectivity determines index value, not column importance.** The same principle from Orders & Payments transfers: high-selectivity predicates benefit; low-selectivity predicates do not.

Check 6: Cross-Layer Integration (University)

A university's `enrolments` table has CHECK (`credits > 0`), a `student_grades` view for advisors (hides disciplinary records), GRANT restricting advisors to the view only, and an index on `student_id`.

An advisor queries `student_grades` for student 4821. How many enforcement layers are exercised?

> All four. **Constraints** guarantee credit values are valid. The **view** hides disciplinary records. **GRANT** prevents the advisor from querying base tables. The **index** on `student_id` accelerates the lookup. Each layer contributes — removing any one weakens the result. The same stack pattern from Orders & Payments transfers to any domain.

Check 7: Enforcement Spectrum Position

Topic 1's enforcement spectrum: engine rejects → validator rejects → nothing rejects.

Where does this topic's enforcement sit?

> **Engine rejects** — completely. The SQL engine rejects invalid data (constraints), rejects unauthorised access (GRANT), and rejects incomplete operations (transaction ROLLBACK). Every mechanism is enforced by the engine, not by application code. The enforcement spectrum table on the next slides fills in all three positions.

Checks 1–7: The Pattern

Check	Domain	What it tests	Lesson
1. Constraint rejection	Library	Three bad inserts	Test is the evidence; success means missing constraint
2. CASCADE vs RESTRICT	Airline	Parent deletion edge case	Wrong cascade silently destroys data
3. View + GRANT	Banking	Trust boundary isolation	View + GRANT = engine-enforced access control
4. Transaction rollback	Ride-sharing	Zero-row UPDATE	Explicit ROW_COUNT check required
5. Index selectivity	Hospital	High vs low selectivity	Selectivity determines index value
6. Cross-layer integration	University	All four layers together	Removing any layer weakens the result
7. Enforcement spectrum	Cross-topic	Engine rejects position	Engine rejects = strongest enforcement

Checks 1–5 isolate single layers across six domains. Check 6 tests composition. Check 7 sets up Topics 6–10.

Revisiting the Three Failures

Failure	Root cause	Fix	DDL
1. SGD 12K duplicate	Missing UNIQUE	`UNIQUE (customer_id, invoice_number)`	One ALTER TABLE
2. Analyst saw everything	Missing trust boundary	Views + GRANT on views only	CREATE VIEW + GRANT
3. Carol's double charge	Missing transaction	BEGIN/COMMIT with ROLLBACK	Three keywords

No mysteries. No magic. Every failure is diagnosable with the vocabulary from this topic, and the fix is a specific DDL statement or transaction wrapper.

The Hardened Schema

The Orders & Payments schema, hardened with all four enforcement layers:

```
┌─ customers ──────────────────────────────────────────────────────┐
│ customer_id PK │ name NOT NULL │ email UNIQUE NOT NULL
├─ orders ─────────────────────────────────────────────────────────┤
│ order_id PK │ customer_id FK NOT NULL (RESTRICT)
│ invoice_number │ UNIQUE(customer_id, invoice_number)
│ status CHECK IN( … ) NOT NULL │ order_total
├─ order_lines ────────────────────────────────────────────────────┤
│ PK(order_id, product_id) │ FK order_id (CASCADE)
│ quantity CHECK > 0 │ unit_price NOT NULL
├─ payments ───────────────────────────────────────────────────────┤
│ payment_id PK │ order_id FK NOT NULL (RESTRICT)
│ amount CHECK > 0 NOT NULL
├─ indexes ────────────────────────────────────────────────────────┤
│ idx_orders_customer │ idx_orders_date
│ idx_orderlines_product
├─ views ──────────────────────────────────────────────────────────┤
│ customer_orders │ revenue_by_product
├─ triggers ───────────────────────────────────────────────────────┤
│ trg_orderline_after_insert (maintains order_total)
└──────────────────────────────────────────────────────────────────┘
```

This is the final relational deliverable — the comparison baseline for Topics 6–10.

The Enforcement Spectrum: Filled In

Topic 1 introduced the spectrum. This topic fills in the "engine rejects" position completely:

Position	What rejects	Mechanism	Topic
Engine rejects	Invalid data, unauthorised access, incomplete operations	Constraints + GRANT + transactions	**Topic 5**
Validator rejects	Invalid documents (after creation)	XSD, JSON Schema	Topic 7
Nothing rejects	Any shape accepted; no validation	Schema-on-read (document model default)	Topic 6

Predict: A colleague adds only indexes (Layer 4) to an unconstrained schema — skipping Layers 1–3. The indexes make queries fast. What can still go wrong?

Speed Without Correctness

> Everything. Fast queries on garbage data return garbage answers quickly. A reporting user with full table access can still see customer emails. A crash mid-checkout still produces double charges.

Speed without correctness is worse than slow correctness — the wrong answers arrive faster, and everyone trusts them because the system "feels" reliable.

The enforcement stack is not optional layers to pick from. It is a dependency chain: each layer requires the one below it.

Concepts That Recur

Concept	Introduced Here As	Recurs In
Trust boundary	Views + GRANT restricting role visibility	Topic 6 (schema-on-read as absent trust boundary), Topic 7 (XSD at organisational boundary)
Enforcement spectrum (filled in)	Engine rejects = constraints + GRANT + transactions	Topic 7 (validator rejects = XSD), Topic 8 (nothing rejects = RDF)
Atomicity	Transaction wrapping multi-step operations	Topic 6 (single-document atomicity in MongoDB)
Selectivity	Index effectiveness depends on predicate specificity	Topic 10 (graph index strategies)
Derived attribute trigger	Engine-maintained computed column	Topic 6 (computed fields in MongoDB aggregation)
Cascade behaviour	ON DELETE CASCADE for weak entities	Topic 10 (DETACH DELETE in property graphs)

Course Map

```
Topic 1 (vocabulary, decision framework, four models + validation surveyed)
    │
    ├── Topics 2–5: Relational deep-dive
    │   ├── Topic 2: ERD — capturing business rules before tables
    │   ├── Topic 3: Normalisation — BCNF, star schema, justified downshifts
    │   ├── Topic 4: SQL — FWGHOS pipeline, JOINs, aggregation, CTEs
    │   └── ▸ Topic 5: Enforcement — constraints, transactions, indexes, access control
    │
    ├── Topics 6–10: Alternative models + cross-cutting discipline
    │   ├── Topic 6: Document (MongoDB) — schema-on-read, embedding vs referencing
    │   ├── Topic 7: Schema validation — XSD and JSON Schema at trust boundaries
    │   ├── Topic 8: RDF — knowledge graphs, global identifiers, open-world assumption
    │   ├── Topic 9: SPARQL — querying knowledge graphs
    │   └── Topic 10: Property graph (Neo4j/Cypher) — bounded traversal
    │
    └── Topic 11: Capstone — multi-model design
```

Topics 1–5 complete the relational foundation. The hardened schema becomes the comparison baseline.

From reading to protecting: Topic 4's FWGHOS pipeline taught you to *read* data from a correct schema. This topic taught you to *protect* that schema — the `revenue_by_product` view encapsulates a multi-table join that Topic 4 students would recognise, but now it is accessible only through a controlled interface.

Next: Topic 6 Document Model

The document model sits at "nothing rejects" by default — documents can have any shape, and the database stores them without validation.

JSON Schema validation can move it toward "validator rejects," but it's opt-in. The opposite of what we just built.

The enforcement spectrum, filled in by this topic, becomes the framework for evaluating every alternative model.

Lab Preview

Five-part lab — hardening the Orders & Payments schema, step by step.

- **Part 1** (~25 min): Add constraints (CHECK, UNIQUE, NOT NULL) and test with bad data
- **Part 2** (~20 min): Choose FK cascade behaviours and verify with edge cases
- **Part 3** (~20 min): Create reporting views, users, and GRANT least privilege
- **Part 4** (~20 min): Wrap multi-step operations in transactions; test rollback
- **Part 5** (~25 min): Add indexes, verify with EXPLAIN, run the full-stack integration test

Tools: Google Colab with MySQL (`%%sql` magic)

Lab file: `05-lab-enforcement.ipynb`